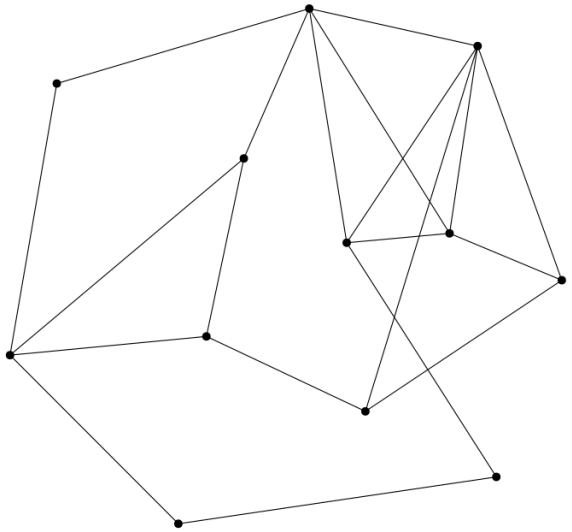
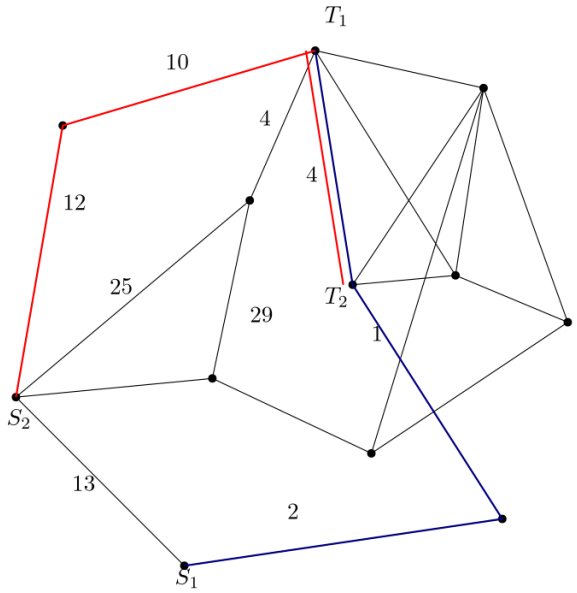
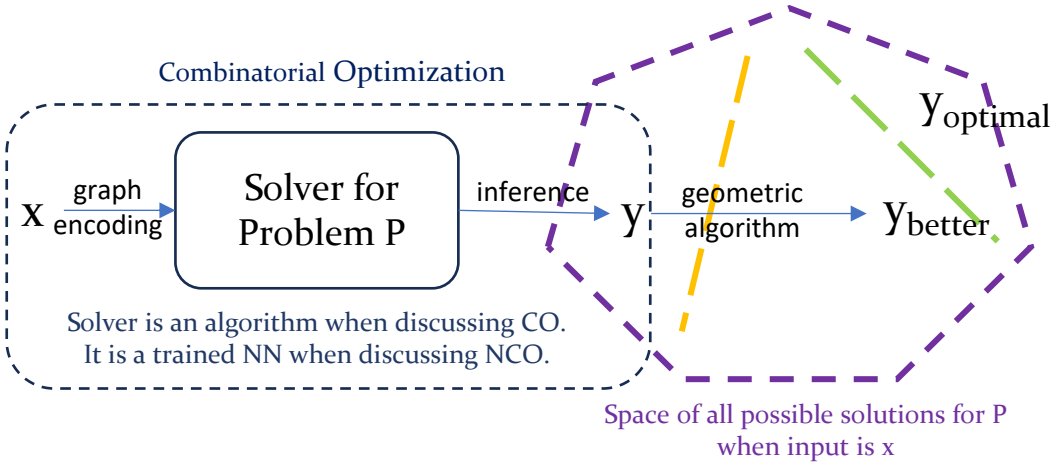
Neural Combinatorial Optimization for Multi Agent Path Finding on Graphs







Graphs

$G = (V, E, w)$

V : vertices

E : edges

w : weight on the edges

Multi Agent Path Finding (MAPF)

$\{A_1, A_2, \dots, A_k\}$: agents

$\{s_1, s_2, \dots, s_k\} \in V$: starting vertices

$\{t_1, t_2, \dots, t_k\} \in V$: target vertices

$\{\pi_1, \pi_2, \dots, \pi_k\}$: paths/policy

NP-hard : $O(n^c)$ algorithm unlikely

Neural Combinatorial Optimization (NCO)

CO : mathematical problem whose solution space is a set of discrete choices

NCO : neural networks learns the problem and given an output from the solution space

Geometric NCO : improves solution obtained by exploiting the structure of solution space

Question

Can we train GNNs to obtain better inference (solution) for MAPF on Graphs using Geometric NCO approach?

Base Papers

1. **Geometric Algorithms for Neural Combinatorial Optimization with Constraints (NeurIPS 2025)**
   Nikolaos Karalias, Akbar Rafiey, Yifei Xu, Zhishang Luo, Behrooz Tahmasebi, Connie Jiang, Stefanie Jegelka
2. ***Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks (SoCS 2019)***
   Roni Stern, Nathan R. Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne T. Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Eli Boyarski, Roman Bart'ak
3. ***Graph Attention Networks based Multi-agent Path Finding via Temporal-spatial Information Aggregation (PLOS One 2025)***
   Qingling Zhang, Peng Wang, Cui Ni, Xianchang Liu

Current Scenario

Extended A* , Conflict Based Searches, Generalized RL algorithms

Plan

Specific variation of MAPF → Obtain a structure from the solution space → Design a good objective function
(Sept 30th) (Sept 30th) (Sept 30th)

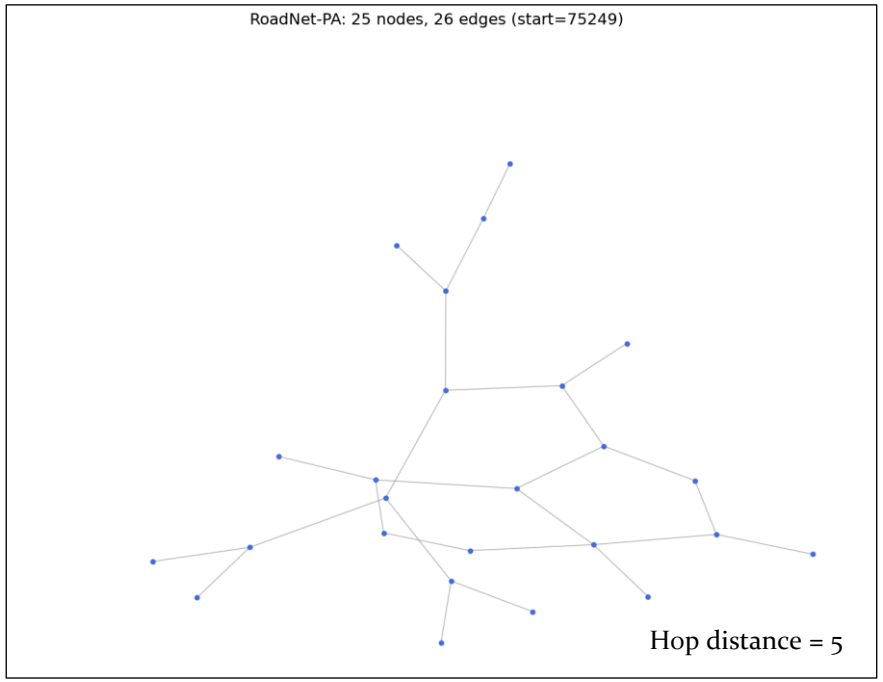
→ Train a GNN → Benchmark (Networks, Heuristic models, Algorithms)
(Oct 30th)

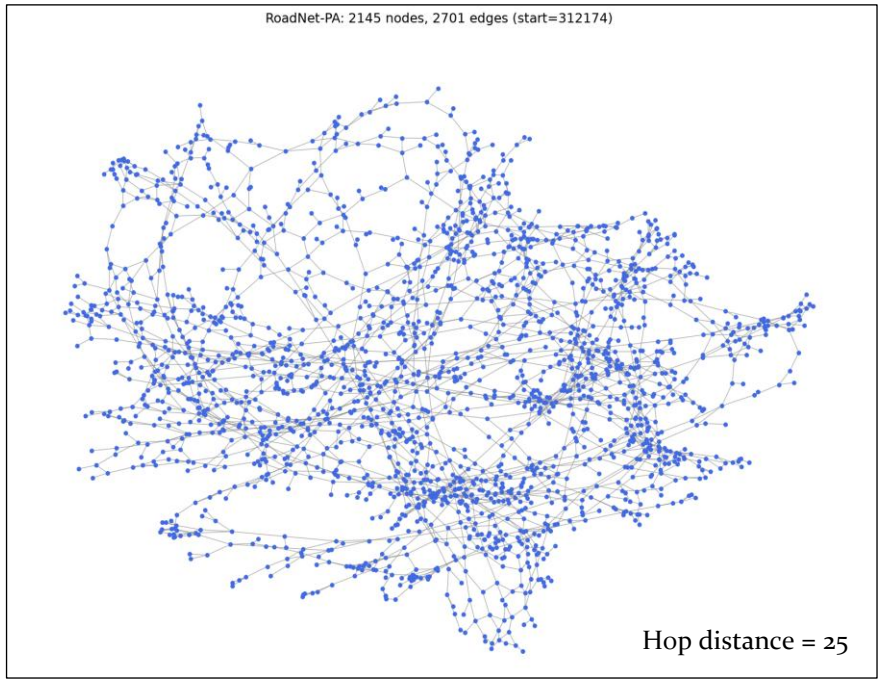
Data Set

1. Synthetic or randomly generated graphs.
2. Real road networks (SNAP, movingai, zenodo)

RoadNet-PA (SNAP)

Representation: Edge List (txt file)
Nodes: 1088092
Edges: 1541898
Average degree: 2.834
Connected components: 206
Diameter: 786





Realization

1. Representation – edge level task – i.e. we have to learn on the edge weights. This seems to be the most suitable as per literature
2. Difficult to carry out on real road networks – they are humongous, not sensible when no. of agents are less. Very difficult to visualize. High compute times.
3. Already Done Work:

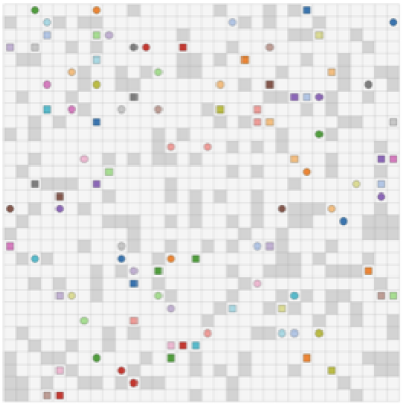
Learned Representations Enhance Multi Agent Path Planning (ICML 2025)
Marius Captar, Herke van Hoof

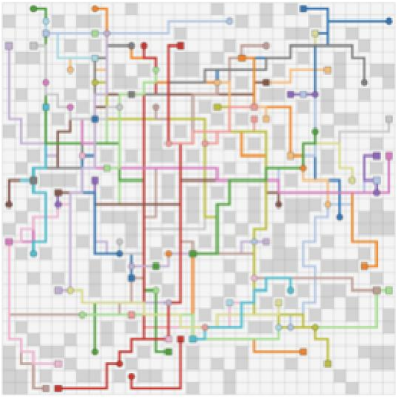
Explanation of the Work

In short: They train a NN to alter the edge weights of the graph that causes a conventional solver to perform better.

- Input Graph: $G = (V, E)$
- Edge usage vector: $y_e = \sum_{i,t} 1\left[\left(v_i^t, v_i^{t+1}\right) = e\right] \quad y \in \mathbb{N}^{|E|}$
- Total Plan Cost Function: $c(w, y) = \sum_{e \in E} w_e\, y_e$
- Discrete Optimization Problem: $y^*(w) = \arg\min_{y \in \mathcal{Y}} c(w, y)$ $\mathcal{Y}$ is set of all plans
- Loss Function: $L\left(\hat{y}, y^*\right) = \frac{1}{M}\sum_{e \in E}\left(\hat{y}_e - y_e^*\right)^2$
- Perturbed Edge Weights: $w' = w + \lambda\frac{\partial L}{\partial \hat{y}}$
- Surrogate Gradient: $\nabla_w f^\lambda(w) = -\frac{1}{\lambda}\left(\hat{y} - y^\lambda\right)$
- Neural Network: $w = \mathrm{NN}_\theta$
- Update to the learned weights so that the heuristic algorithm (called Prioritized Planner) can perform better
- Two methods: Edge Model *(independent learning in each individual edge) and*
  GNN Model *(two-layer GraphSAGE that takes in context of whole graph and agent positions)*
- Compared against EECBS algorithm *(heavy but more accurate).*

random-32-32-20 map





Plans

1. Check , incase, there are more work done already.
2. Check the same technique for Make-span version of MAPF on grids. (By 18th Oct)
3. Visualize the solution space for Grid Maps. (By 18th Oct)

